

OPENAGENCY: SELF-EVOLVING MULTI-AGENT SYSTEM VIA GOVERNED COLLABORATION PROTOCOLS

Anonymous authors

Paper under double-blind review

ABSTRACT

Large language model (LLM) agents have shown strong capabilities on complex tasks, encouraging multi-agent systems that coordinate specialized roles. Improving how these agents collaborate requires flexibility, so that coordination decisions are still made at runtime, together with controllability, so that they can be inspected and revised afterwards. Existing systems supply only one of the two, because a predefined workflow fixes the execution route while free-form communication leaves the realized coordination opaque. In this paper, we introduce OpenAgency, a self-evolving multi-agent system built on governed collaboration protocols. A protocol defines roles, admissible handoffs, handoff modes, and completion conditions, and each agent still selects its next collaborator inside the admissible set, which turns the collaboration pattern itself into an evolvable object. To evolve that object, OpenAgency keeps an immutable trace that retains rejected handoffs and failed completions alongside successful events, and a collaboration-driven evolver converts this evidence into a bounded number of atomic protocol edits, each screened by a structural validity operator and by an evolution agent that scores the realized coordination. Experiments on fourteen benchmarks show that OpenAgency achieves the best primary metric on every benchmark, covering general question answering, long-horizon tasks, and production-oriented tasks. These results show that a governed protocol is an effective object for multi-agent adaptation.

1 INTRODUCTION

Large language models (LLMs) (OpenAI, 2023; 2022) have shown strong capabilities on complex tasks such as software engineering (Yang et al., 2024b; Wang et al., 2024b; Zhang et al., 2024a), data research (Liu et al., 2026; Li et al., 2024), and long-horizon planning (Zhang et al., 2026; Xu et al., 2024). Multi-agent systems built on LLMs extend these capabilities by assigning complementary roles that exchange intermediate evidence (Wang et al., 2024a; Xi et al., 2023; Hong et al., 2024; Qian et al., 2024; Wu et al., 2023), which lets one system plan and verify the same task through cooperation. However, as tasks become longer and more heterogeneous, the collaboration process itself becomes both a source of errors and an underused source of improvement.

Evolving the way agents collaborate requires flexibility and controllability at the same time. Flexibility matters because a multi-agent system, unlike a single agent that revises its own reasoning inside one context, must recover what went wrong across roles, and a system that cannot repair a failed handoff at runtime loses the remainder of the run along with it. Controllability is equally necessary, since an update is informative only when the system can tell which coordination step was responsible and can verify that the revised collaboration remains admissible. Once either property is missing, cross-run improvement falls back to a blind search driven by the final score. For instance, on SWE-bench Verified (Jimenez et al., 2024), a free-form multi-agent baseline reaches only 51.4% file match and an SOP-based baseline reaches 59.8%, far below the 86.2% attained once the collaboration itself can be revised from execution traces (Table 2).

Existing collaboration representations hold only one of these two properties. *Blueprint-driven* systems encode a graph, workflow, or standard operating procedure (Hong et al., 2024; Wu et al., 2023; Fourney et al., 2024), and recent methods automate this design through workflow search or generated agent programs (Zhang et al., 2025b; Hu et al., 2025; Zhang et al., 2025a). Such designs are

controllable, yet each selected workflow still determines the exact route taken during execution, and a runtime deviation therefore cannot be repaired locally. *Agent-driven* systems let debate, dialogue, or dynamic connectivity emerge at runtime (Li et al., 2023; Du et al., 2023; Liu et al., 2024; Park et al., 2023), and a learned orchestrator can further reorder agent activations from reward (Dang et al., 2025). Flexibility here costs transparency, since the realized coordination stays opaque and is hard to attribute or revise across runs. One line of work fixes the route and the other hides it, so neither yields a representation that is checkable at runtime and revisable from experience.

In this paper, we aim to make the collaboration pattern itself evolvable by holding flexibility and controllability in one representation. To this end, we introduce *OpenAgency*, a self-evolving multi-agent system built on *governed collaboration protocols*. As illustrated in Figure 1, OpenAgency represents a protocol as a tuple $\mathcal{P} = (\mathcal{R}, \mathcal{N}, \alpha, \mathcal{H}, \mu, \Omega, \mathcal{C})$ whose components fix the roles and the admissible handoffs between them. The admissible handoff set $\Gamma(n, s)$ carries both properties in a single object, since it rules out inadmissible transitions while the choice inside it is left to the agent at runtime. Because every coordination decision is expressed in this object and its legality is decidable, the collaboration pattern becomes revisable between runs. Building on this, OpenAgency records accepted handoffs, rejected handoffs, completion events, and failures in an immutable trace, and a collaboration-driven evolver turns this evidence into at most k atomic edits over the protocol. A validity operator Π_V filters malformed topologies and an evolution agent scores the coordination each surviving candidate realizes, so every deployed round stays admissible and traceable to the events that motivated it.

Experiments across fourteen benchmarks covering general question answering, long-horizon tasks, and production-oriented tasks show that OpenAgency achieves the best primary metric on every benchmark, lifting the unstructured multi-agent initialization by up to 54.6 absolute points (Table 1). To the best of our knowledge, OpenAgency is the first multi-agent system in which the collaboration pattern is an evolvable object, updated from immutable execution traces without regenerating the whole workflow. We summarize our contributions as follows.

- **Evolvable Collaboration Protocol:** We make the collaboration pattern itself a governed and evolvable object, because the admissible handoff set is checkable before a transition is taken and revisable after the run that produced it.
- **Collaboration-driven Evolution:** We propose an evolver that learns from the coordination evidence prior systems discard, namely rejected handoffs and failed completions, and converts it into at most k atomic edits over protocol structure, predicates, and shared context. A validity operator and a collaboration quality screen keep every deployed round admissible and free of regression.
- **Strong Performance:** OpenAgency reaches the best primary metric on all fourteen benchmarks (Section 4), and the same advantage holds against free-form and SOP collaboration substrates under an identical executor. Ablations further verify that both event content and temporal order are necessary for protocol evolution.

2 RELATED WORK

Multi-agent Collaboration. Existing systems expose collaboration with different degrees of structure. Blueprint-driven approaches encode graphs, workflows, or standard operating procedures (Hong et al., 2024; Qian et al., 2024; Wu et al., 2023). These systems make coordination explicit but typically fix the selected topology before execution. Agent-driven systems allow collaboration to emerge from debate (Du et al., 2023; Liang et al., 2023), role-conditioned dialogue (Li et al., 2023; Chan et al., 2023), or dynamic connectivity (Liu et al., 2024; Zhang et al., 2024b). Collaboration then adapts during execution, though the realized structure remains difficult to inspect and replay. Recent methods also optimize communication for accuracy or efficiency (Sun et al., 2023; Chen et al., 2024a;b). Across both families, the collaboration pattern is treated as a fixed design choice or as an emergent byproduct, and neither case leaves an object that later runs can update.

Agent Self-evolution. Agent systems can improve through reasoning feedback, memory, prompt optimization, or program search. Reflexion and Self-Refine use rollout feedback to revise subsequent reasoning (Shinn et al., 2023; Madaan et al., 2023), while AgentGym and experiential co-learning accumulate experience across interactions (Xi et al., 2024; Qian et al., 2023). DSPy, TextGrad, and OPRO optimize language-model programs or prompt strings (Khattab et al., 2024; Yuksekogonul et al., 2024; Yang et al., 2024a). More recently, AFlow, automated agent design, and

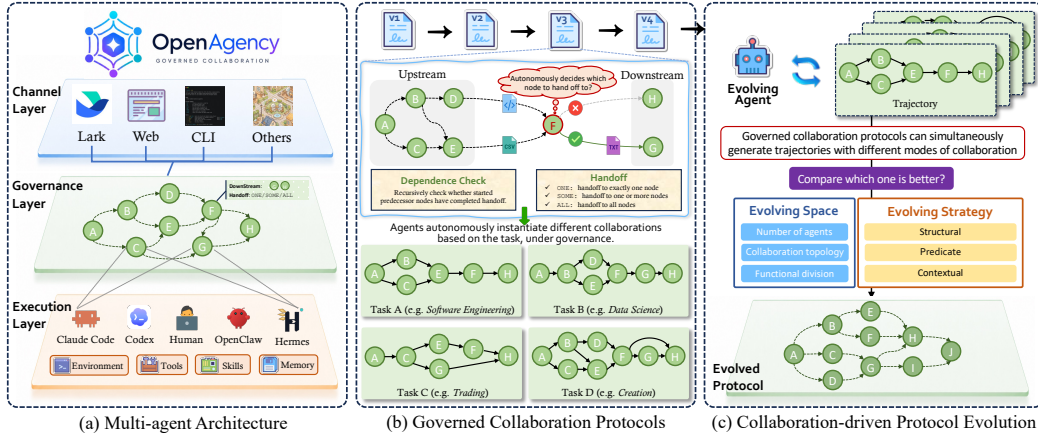


Figure 1: Overview of OpenAgency. (a) Three layers keep the collaboration protocol independent of the backend that realizes each role. (b) Under one governed protocol the agents choose their own handoffs, so different tasks instantiate different collaborations. (c) The evolving agent reads the collected trajectories and returns the next protocol, moving inside the evolving space under one of three edit strategies.

MaaS search over workflow graphs or generated agent programs (Zhang et al., 2025b; Hu et al., 2025; Zhang et al., 2025a), while evolving orchestration trains a central controller to reorder agent activations (Dang et al., 2025). These methods show that collaboration structure can be optimized, but what they return is an executable topology or a controller policy, neither of which exposes a boundary that a later round can check a candidate revision against.

In this work, we study which representation of collaboration can support evolution at all. OpenAgency defines a governed protocol that fixes admissible handoffs and completion conditions without prescribing one route, so agents keep local choices during execution while every decision remains checkable, which is what makes the protocol revisable from immutable traces across runs. OpenAgency therefore evolves the governance boundary that admits runtime decisions, while workflow search and orchestrator learning optimize the executed route or the policy that produces it.

3 OPENAGENCY

OpenAgency treats collaboration as a first-class object separated from the executors that realize each role. It combines a layered architecture (Section 3.1), a governed protocol that makes coordination inspectable (Section 3.2), and bounded protocol evolution driven by execution traces (Section 3.3), as Figure 1 illustrates.

3.1 MULTI-AGENT ARCHITECTURE

To apply one collaboration protocol across applications with heterogeneous executors, OpenAgency separates multi-agent collaboration into three layers that keep the protocol independent of backend-specific prompts and actions.

Channel Layer. The channel layer exposes the collaboration to its users through an enterprise chat workspace, a web console, or a command line, and keeps instructions, agent messages, human feedback, and coordination events on one shared timeline.

Governance Layer. The governance layer hosts the collaboration protocol and validates each proposed handoff against the current protocol state. It records both accepted and rejected events in an immutable trace that supplies the evidence for later protocol evolution.

Execution Layer. The execution layer binds each role to a concrete executor. An executor can be an LLM agent, a coding harness, a domain-specific tool, or a human reviewer, and it sits below the governance layer, so the protocol constrains how roles hand off whichever backend realizes a role.

We formalize a running OpenAgency instance at step t as $\mathcal{I}^{(t)} = (\mathcal{X}^{(t)}, \mathcal{P}, \mathcal{E}, s^{(t)})$, where $\mathcal{X}^{(t)}$ is the shared context, $\mathcal{P}$ is the governed protocol, $\mathcal{E}$ is the set of available executors, and $s^{(t)}$ is the current protocol state. The map $\rho : \mathcal{R} \rightarrow \mathcal{E}$ binds each role in the role set $\mathcal{R}$ to an executor and stays separate from the protocol logic. The governance layer renders the shared state into a role-specific execution context $u^{(t)} = f_{\text{ctx}}(\mathcal{X}^{(t)}, s^{(t)}, n, r, \mathcal{P})$ for the active node n with assigned role r , since an LLM executor may require tool instructions while a human executor may require a concise protocol description. Adapters return messages, handoffs, finishes, and failures in a common event representation, so the executor bound to a role can change without changing the handoff rules, trace format, or completion predicates.

3.2 GOVERNED COLLABORATION PROTOCOLS

To govern collaboration without fixing a complete execution route, OpenAgency represents collaboration as a declarative tuple that exposes enough structure to validate each handoff while preserving the agent’s choice among admissible targets.

Protocol Definition. OpenAgency represents a governed collaboration protocol as

$$\mathcal{P} = (\mathcal{R}, \mathcal{N}, \alpha, \mathcal{H}, \mu, \Omega, \mathcal{C}), \quad (1)$$

where $\mathcal{R}$ is the set of roles, $\mathcal{N}$ the set of collaboration nodes, $\alpha : \mathcal{N} \rightarrow \mathcal{R}$ assigns each node to a role, $\mathcal{H} \subseteq \mathcal{N} \times \mathcal{N}$ defines admissible handoff edges, $\mu(n) \in \{\text{ONE}, \text{SOME}, \text{ALL}\}$ is the handoff mode, Ω_n is the completion predicate, and $\mathcal{C}$ contains the shared rule and culture that every role sees.

Admissible Handoff Set. For a node n under state s , the admissible handoff set is $\Gamma(n, s) = \{n' \in \mathcal{N} \mid (n, n') \in \mathcal{H} \text{ and the transition is valid under } s\}$. The handoff mode $\mu(n)$ then determines how the active agent selects targets from $\Gamma(n, s)$. Under ONE the agent picks exactly one admissible target, under SOME a nonempty subset, and under ALL every admissible target. A node with no admissible target completes without a handoff, and the local choice within $\Gamma(n, s)$ is left to the agent.

Handoff Validation and Completion. When the agent proposes a handoff (n, B, m) with target set B and message m , the protocol validates B under $\mu(n)$. An admissible handoff commits B as pending targets and updates the state, and an inadmissible one leaves the state unchanged and returns structured feedback so the agent may propose another target. Committing a target set does not activate it. A finish event (n, y) confirms that the source node has completed its work, and the governance layer activates the committed targets only when $\Omega_n(y)$ holds, so downstream execution waits for the completion predicate even after an admissible target set is committed. Activation runs one further dependence check, which walks back from each committed target and confirms that every started predecessor of that target has already handed off, so a node begins only once the collaborators it depends on have delivered.

Algorithm 1 states the resulting runtime interpreter. The state advances only on admissible handoffs and valid completions, while the trace keeps every event, including the ones the boundary refused, so later evolution reads both successful and unsuccessful coordination.

3.3 COLLABORATION-DRIVEN PROTOCOL EVOLUTION

Given a protocol whose decisions are explicit and checkable, the remaining question is how to revise it from experience. Revising a protocol takes the shape of an optimization problem, since it needs a signal that says where the current object is wrong, a space of admissible moves, and a rule that decides which move to keep. The final answer alone cannot indicate which of a run’s many coordination events should change, and regenerating the whole workflow each round would destroy the governance boundary that makes a change inspectable. A single evolving agent therefore drives the loop, reading a search direction off the collaboration trace, moving inside a bounded space of protocol edits, and keeping a move only when it survives an admissibility and quality screen.

Trace Diagnosis as Search Signal. Each execution produces an ordered trace $\tau = (e_1, s_1), \dots, (e_T, s_T)$ of coordination events with the resulting states, where an event e_t is one of accepted handoff, rejected handoff, finish, message, or failure. Unlike prior systems that keep only the successful trajectory or the final score, the trace retains rejected handoffs and failed completions,

Algorithm 1 Collaboration under a Governed Protocol

Input: Protocol $\mathcal{P}$, initial context $\mathcal{X}$, executor mapping $\rho: \mathcal{R} \rightarrow \mathcal{E}$
Output: Immutable collaboration trace τ

```

1:  $\tau \leftarrow \emptyset$ ; activate start node  $n_0$ ; obtain initial state  $s$ 
2: while active nodes exist do
3:   for each active node  $n$  with role  $r = \alpha(n)$  do
4:      $e \leftarrow \rho(r)(f_{\text{ctx}}(\mathcal{X}, s, n, r, \mathcal{P}))$  ▷ executor proposes an event
5:     if  $e$  is a handoff  $(n, B, m)$  and  $B$  is admissible then
6:       commit  $B$  as pending targets and update  $s$ 
7:     else if  $e$  is a finish  $(n, y)$  and  $\Omega_n(y)$  holds then
8:       mark  $n$  complete, then activate every committed target whose started predecessors have handed
      off
9:     else
10:      return structured feedback to  $\rho(r)$  ▷ refused at the governance boundary
11:      append  $(e, s)$  to  $\tau$  ▷ accepted and refused events are both retained
12: return  $\tau$ 

```

which record the transitions the collaboration attempted and could not take. The evolving agent runs an analyzer f_{diag} over $\tau^{(1:v)}$ and returns the coordination sites that account for the observed failures, namely the handoff edges the agents repeatedly attempted and the admissible set refused, together with the completion predicates that fired without a usable output. This signal plays the role a gradient plays in parameter optimization, since it attributes a run-level outcome to individual coordination decisions and points at the part of the protocol a revision should touch.

Factorized Evolving Space. The search space factorizes into three levels. *Number of agents* covers the node count $|\mathcal{N}|$. *Collaboration topology* covers the handoff edge set $\mathcal{H}$ and the handoff modes μ . *Functional division* covers the role assignment α , the completion predicates Ω , and the shared rule and culture in $\mathcal{C}$, which together fix what each node owes the collaborators downstream of it. A point in this space is not one execution route, because each agent still chooses inside $\Gamma(n, s)$, so a single protocol unfolds into many realized collaboration patterns at inference time, as the four task instantiations of Figure 1 show, and the space stays small while the behavior it covers does not. The factorization also makes comparison readable, since two candidates that differ on one level agree everywhere else, so an overloaded role can be told apart from a missing edge.

Bounded Protocol Optimizer. A move in this space is a composite edit of at most k atomic changes drawn from three edit strategies. A *structural* edit adds or removes a node or a handoff edge, a *predicate* edit rewrites a completion or transition condition, and a *contextual* edit rewrites the rule and culture a role reads. At round v the optimizer f_{evolve} takes one such step from the incumbent under the analyzed trace signal:

$$\hat{\mathcal{P}}^{(v+1)} = f_{\text{evolve}}(\mathcal{P}^{(v)}, f_{\text{diag}}(\tau^{(1:v)})), \quad |\hat{\mathcal{P}}^{(v+1)} \ominus \mathcal{P}^{(v)}| \leq k. \quad (2)$$

We set $k = 4$, which bounds the step the way a learning rate bounds a parameter update and keeps each candidate a small inspectable change. The strategy is a bounded greedy ascent, proposing one candidate per round and retaining the incumbent whenever the candidate fails the screen below, so the deployed sequence never moves downhill in collaboration quality.

Admissibility and Quality Screen. A step is screened twice before it is deployed. The first check is a structural validity operator $\Pi_{\mathcal{V}}$ on the candidate topology, which rejects malformed handoff edges, unresolved role references, duplicate node identifiers, and completion states no path can reach, so a candidate that is not a well-formed protocol never runs. Writing $\tilde{\mathcal{P}}^{(v+1)} = \Pi_{\mathcal{V}}(\hat{\mathcal{P}}^{(v+1)})$ for the validity-checked candidate, the second check runs it and hands the resulting traces back to the evolving agent, which returns a collaboration quality score q_{col} from the realized coordination, reading whether each role delivered what its successors required and whether the completion conditions closed on a usable output. The deployment rule is

$$\mathcal{P}^{(v+1)} = \begin{cases} \tilde{\mathcal{P}}^{(v+1)}, & \tilde{\mathcal{P}}^{(v+1)} \neq \perp \wedge q_{\text{col}}(\tilde{\mathcal{P}}^{(v+1)}) \geq q_{\text{col}}(\mathcal{P}^{(v)}), \\ \mathcal{P}^{(v)}, & \text{otherwise,} \end{cases} \quad (3)$$

so a candidate is deployed only when it is well formed and does not degrade the observed coordination. Both checks read the candidate protocol and the traces it produces, which keeps the update loop closed over collaboration evidence.

4 EXPERIMENTS

We evaluate the governed protocol of Section 3 on fourteen benchmarks (Section 4.1) under one executor and tool budget (Section 4.2), and report the main comparison in Section 4.3. Section 5 then locates where the gain comes from, through the round-wise effect of protocol evolution (Section 5.1), the advantage of multi-agent protocols (Section 5.2), the trace signal the evolver consumes (Section 5.3), the comparison against free-form and SOP substrates (Section 5.4), and the levels of the evolution space that move the metric (Section 5.5), with the activity of the governance boundary reported in Appendix C.4.

4.1 BENCHMARKS

We evaluate OpenAgency on fourteen benchmarks organized into three categories. *General question answering* contains HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), and HotpotQA (Yang et al., 2018), and stresses whether protocol adaptation helps when each instance admits a short answer. *Long-horizon tasks* contains SWE-bench Verified (Jimenez et al., 2024), DeepPlanning (Zhang et al., 2026), and the text-only subset of Humanity’s Last Exam (HLE) (Phan et al., 2025), and stresses whether governed handoffs support dependent decisions. *Production-oriented tasks* contains DevEval (Li et al., 2024), DDR-Bench (Liu et al., 2026), TheAgentCompany (Xu et al., 2024), MARBLE (Zhu et al., 2025), AutomationBench (Shepard & Salimans, 2026), and OfficeQA Pro (Opsahl-Ong et al., 2026), and stresses whether one protocol representation applies across heterogeneous tools and task surfaces. Appendix A describes each benchmark.

4.2 EXPERIMENTAL SETUP

Governed collaboration and its evolution have to be separated from the executor’s own capability, so we use GPT-5.5 as the underlying language model across all benchmarks, and each role receives the tool surface its benchmark requires. For each benchmark, we compare three in-house conditions under an identical model and tool budget, namely a single-agent ablation with one role and no handoff, the unstructured multi-agent initialization v_0 , and the evolved protocol after four rounds (v_4). We report all three against a prior reference (Prior Ref.), a strong baseline from prior work that fixes the absolute level of the benchmark, taken as the strongest externally reported model or agent and as a same-model direct-LLM baseline where no stronger external system is reported. Model, temperature, and tool budget stay fixed across rounds, and only the protocol template changes. Table 1 reports the final round, and Appendix D lists all four.

4.3 MAIN RESULTS

Capability on General Question Answering. The five single-turn benchmarks (HumanEval, MBPP, GSM8K, MATH, HotpotQA) probe *short-chain reasoning capabilities*, since each instance admits a single well-defined answer and leaves little room to coordinate. Under a common GPT-5.5 executor, we compare OpenAgency with its two ablations and each benchmark’s strongest prior reference. As shown in Table 1, OpenAgency obtains the best primary metric on all five benchmarks, reaching 100.0 pass@1 on HumanEval and lifting MATH from 32.4 to 87.0 over the initialization. Agent count alone does not produce this gain, since the unstructured initialization trails the single-agent ablation on HotpotQA at 64.0 against 75.9 F1 while the governed protocol carries the same benchmark to 82.3. Evolution prunes handoffs whose upstream output the trace repeatedly flags as ineffective, and each benchmark settles on a tight solve-critique-revise loop matched to its dominant failure mode, so the protocol turns a short chain into a verified one.

Capability on Long-horizon Tasks. SWE-bench Verified, DeepPlanning, and HLE stress *long-horizon reasoning capabilities*, since each instance requires several dependent decisions and a bad handoff at any stage can invalidate the entire run. We compare OpenAgency with a monolithic single-agent solver and each benchmark’s strongest prior reference. Table 1 reports a lead on all three, reaching 86.2 file match on SWE-bench Verified against a prior reference of 42.8 and 37.7 accuracy on HLE against 18.0. What produces the lead is visible in the protocol each benchmark converges to. On SWE-bench the evolver installs a retry edge when a patch fails and splits an overloaded Verifier into structural and semantic components. The DeepPlanning protocol hands

Table 1: OpenAgency and its ablations, listed against each benchmark’s prior reference, a strong baseline from prior work that fixes the absolute level of the benchmark. Single-agent is the single-agent ablation, Multi-agent (init) is the unstructured initialization, and OpenAgency is the evolved protocol. The three run under an identical GPT-5.5 executor and tool budget, and each row uses the benchmark’s native metric.

Benchmark	Metric	Prior Ref.	Single-agent	Multi-agent (init)	OpenAgency
<i>General QA Tasks</i>					
HumanEval	pass@1	92.2	87.5	96.9	100.0
MBPP	pass@1	69.3	73.5	73.5	80.9
GSM8K	EM	97.6	97.2	97.4	98.0
MATH	EM	80.3	12.9	32.4	87.0
HotpotQA	F1	76.4	75.9	64.0	82.3
<i>Long-horizon Tasks</i>					
SWE-bench Verified	FM	42.8	75.6	67.0	86.2
DeepPlanning	Case Acc	46.9	45.0	33.8	54.2
HLE (text-only)	Accuracy	18.0	18.4	5.6	37.7
<i>Production-oriented Tasks</i>					
DevEval	AT	62.5	62.5	82.1	96.9
DDR-Bench	Overall	69.1	42.8	44.0	76.0
TheAgentCompany	Score	41.5	46.3	41.5	53.7
MARBLE	Overall	79.3	78.4	50.0	98.9
AutomationBench	Acc	42.5	23.7	23.6	54.8
OfficeQA Pro	Acc@1%	57.1	35.3	21.1	69.9

a dedicated Verifier the user-specified constraints one at a time, turning a brittle end-to-end plan into a chain of locally verifiable handoffs. Verification on HLE becomes conditional, routing high-confidence answers straight to completion and sending low-confidence ones back through a retry edge. All three shapes are reached from the same initialization, so it is the handoff and completion rules that adapt when local errors are unrecoverable.

Capability on Production-oriented Tasks. The six production-oriented benchmarks (DevEval, DDR-Bench, TheAgentCompany, MARBLE, AutomationBench, OfficeQA Pro) test *cross-domain deployment capabilities*, since each exposes a different tool surface and task format. Measured against each benchmark’s strongest prior reference and against its own ablations, OpenAgency leads on all six in Table 1, improving over that reference by 34.4 points on DevEval, 19.6 points on MARBLE, and 12.8 points on OfficeQA Pro. The template behind these numbers differs per benchmark, from a Reader-Solver-Verifier chain on OfficeQA Pro to a Generator-Critic-Reviser pipeline on MARBLE, and two cases expose what the governance layer contributes. On DDR-Bench, an Auditor role introduced during evolution forces every quantitative finding in the trace to be reflected in the final report, fixing a class of silent omission that free-form baselines leave undetected. The admissible-set constraint on MARBLE keeps each handoff terminating under a protocol that would otherwise allow any-to-any messaging. In both cases the governance layer supplies a constraint the task format alone does not.

5 ANALYSIS

5.1 EFFECTIVENESS OF PROTOCOL EVOLUTION

To verify that bounded protocol edits keep improving collaboration after the first revision, we track the primary metric from the unstructured multi-agent initialization v_0 to the evolved rounds v_1 through v_4 on all fourteen benchmarks. As shown in Figure 2 and Table 7, the evolved protocol improves the primary metric over the initialization on every benchmark, reaching 54.6 points on MATH and 48.9 points on MARBLE. The round-to-round trajectory is non-decreasing on every benchmark, which is the behavior the quality screen enforces, so revision keeps improving the governed protocol after the first round.

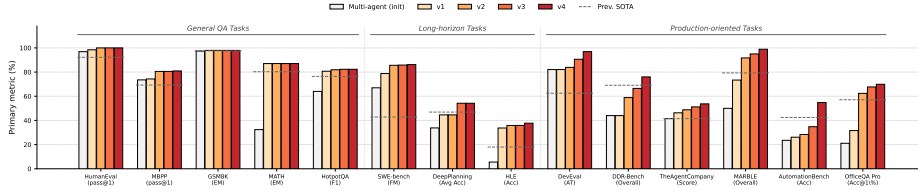


Figure 2: Round-wise trajectory of the primary metric from the multi-agent initialization v_0 through the evolved rounds v_1-v_4 . Grey dashed lines mark each benchmark’s prior reference.

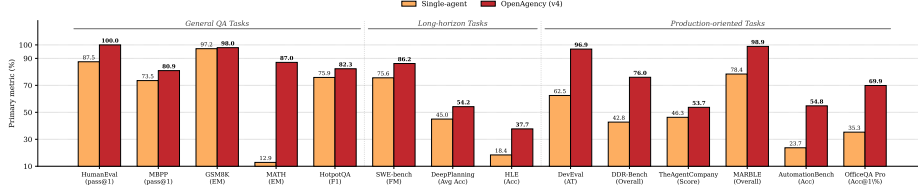


Figure 3: OpenAgency versus its single-agent ablation under identical model and tool budget, isolating governed collaboration from raw solver capacity.

5.2 ADVANTAGE OF MULTI-AGENT PROTOCOLS

OpenAgency attributes its gain to governed collaboration, though a multi-agent system can also improve through sheer agent count. To separate the two, we compare it with its single-agent ablation under an identical GPT-5.5 executor and tool surface. As shown in Figure 3, OpenAgency exceeds the single-agent ablation on every benchmark, with the widest gaps on MATH, OfficeQA Pro, and DevEval. A single node interleaves solving and checking inside one context, while the governed protocol routes a partial output to a downstream role whose completion predicate validates it independently. Read together with the unstructured-initialization column of Table 1, the gain traces to the protocol design, since the same agent budget without governance does not reach the same level.

5.3 ABLATION OF THE TRACE SIGNAL

To identify which part of the trace the evolver actually consumes, we compare the default *Full* evolver against a *Label only* variant that replaces events with a scalar success/failure label and a *Shuffled trace* variant that keeps the events but breaks their temporal order. As shown in Figure 4, both variants lower the final metric on every benchmark in the comparison, with *Shuffled trace* producing the larger drop. Event content locates the coordination step that failed and temporal order records how the failure developed, and neither can be recovered from the other, so the evolver needs a trace that preserves both.

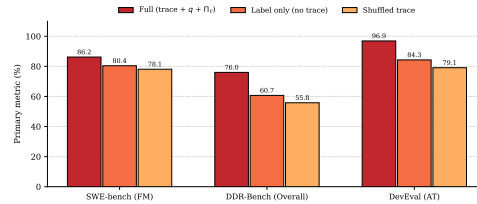


Figure 4: Ablating the trace signal fed to the evolver against a scalar-label variant and a temporally-shuffled variant. The comparison identifies which part of the trace evolution requires.

5.4 COMPARISON AGAINST FREE-FORM AND SOP BASELINES

OpenAgency keeps the collaboration structure explicit and revisable, while free-form systems leave it implicit and standard-operating-procedure systems fix it at design time. To compare the three substrates directly, we evaluate the free-form message-exchange baseline AutoGen (Wu et al., 2023) and the SOP baselines ChatDev (Qian et al., 2024) and MetaGPT (Hong et al., 2024) on four benchmarks. As shown in Table 2, OpenAgency obtains the highest score on all four benchmarks, exceeding the strongest SOP baseline by 26.4 points on SWE-bench Verified. The free-form baseline is the weakest everywhere, and the SOP baselines recover part of the gap because the collaboration

Table 2: OpenAgency against the free-form and SOP baselines.

System	SWE-bench	DevEval	DDR-Bench	TheAgentCompany
AutoGen	51.4	68.9	39.2	38.7
ChatDev	57.6	78.3	44.1	41.9
MetaGPT	59.8	79.5	46.8	42.3
OpenAgency	86.2	96.9	76.0	53.7

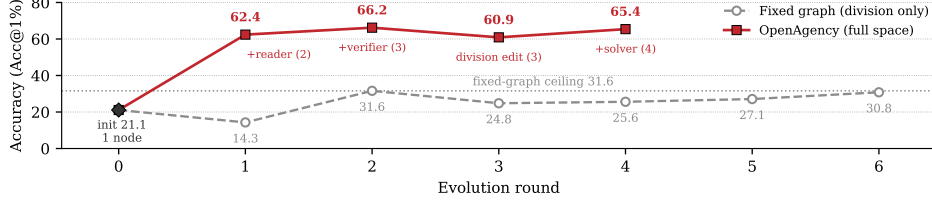


Figure 5: Evolution on OfficeQA Pro under a fixed collaboration graph against evolution over the full space, from one initialization and scored on the same 133 tasks. The fixed arm edits only the functional division, while the full arm may also change the number of agents and the topology.

order is enforced, yet their procedure is fixed at design time and cannot absorb trace-level failure signals, so the remaining gap tracks how much of the collaboration structure stays revisable after deployment.

5.5 ADVANTAGE OF GOVERNED COLLABORATION PROTOCOLS FOR EVOLUTION

To measure how much the collaboration protocol governs what evolution can reach, we split the complete evolution record on OfficeQA Pro into two arms that share one initialization. The fixed arm holds the graph at the single node the initialization produced, which leaves the functional division as its only editable level and reproduces how a standard multi-agent system tunes a prompt over a hand-designed structure, while the full arm is the governed protocol, whose optimizer may also change the number of agents and the collaboration topology. Every plotted point was proposed and executed, and no fixed-arm round became the incumbent.

Figure 5 shows six functional-division rewrites moving the metric between 14.3 and 30.8 without passing the 31.6 of round 2. Widening the graph moves it at once, since one added reader node reaches 62.4 and one added verifier node reaches 66.2. The full arm oscillates as well, because its third round edits the functional division and falls to 60.9, which makes oscillation a property of the search itself. A frozen graph keeps the search inside a level the trace already reports as saturated, while one admissible structural edit clears the fixed ceiling by more than thirty points. The deployed incumbent is the round-2 protocol of the full arm, whose stable rerun is the v_4 checkpoint of Table 1.

To check whether the structural level is the one that always pays, we read the same record on two further benchmarks. On DDR-Bench and DevEval the optimizer kept the two-node graph it started from and carried the metric from 44.0 to 76.0 and from 82.1 to 96.9 through functional-division edits alone, while on OfficeQA Pro no amount of functional-division rewriting substituted for a structural edit. Fixing the graph at design time therefore wagers that the structural level never carries the gain, a wager that is idle on these two benchmarks and costs more than thirty points on the third, while a governed protocol makes that choice at run time from the level the trace reports as saturated.

6 CONCLUSION

In this paper, we introduce OpenAgency, a self-evolving multi-agent system built on governed collaboration protocols. By constraining runtime handoffs while preserving each agent’s choice among admissible collaborators, OpenAgency makes the collaboration pattern an evolvable object and turns ordered traces, including the rejected handoffs prior systems discard, into bounded protocol updates filtered by a validity operator and a quality screen. Across fourteen benchmarks it reaches the best primary metric on each, above the strongest prior reference and its own ablations.

REFERENCES

- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. Chateval: Towards better LLM-based evaluators through multi-agent debate, 2023. URL <https://arxiv.org/abs/2308.07201>.
- Justin Chih-Yao Chen, Swarnadeep Saha, and Mohit Bansal. Reconcile: Round-table conference improves reasoning via consensus among diverse llms, 2024a. URL <https://arxiv.org/abs/2309.13007>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>.
- Weize Chen, Jiarui Yuan, Chen Qian, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Optima: Optimizing effectiveness and efficiency for LLM-based multi-agent system, 2024b. URL <https://arxiv.org/abs/2410.08115>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukas Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Yufan Dang, Chen Qian, et al. Multi-agent collaboration via evolving orchestration. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2505.19591>.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023. URL <https://arxiv.org/abs/2305.14325>.
- Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. Magentic-one: A generalist multi-agent system for solving complex tasks, 2024. URL <https://arxiv.org/abs/2411.04468>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Track on Datasets and Benchmarks*, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework, 2024. URL <https://arxiv.org/abs/2308.00352>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2025. URL <https://arxiv.org/abs/2408.08435>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations (ICLR)*, 2024.

- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society, 2023. URL <https://arxiv.org/abs/2303.17760>.
- Jia Li, Ge Li, Yunfei Zhao, Yongmin Li, Huanyu Liu, Hao Zhu, Lecheng Wang, Kaibo Liu, Zheng Fang, Lanshen Wang, Jiazheng Ding, Xuanming Zhang, Yuqi Zhu, Yihong Dong, Zhi Jin, Binhua Li, Fei Huang, and Yongbin Li. DevEval: A manually-annotated code generation benchmark aligned with real-world code repositories. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. URL <https://arxiv.org/abs/2405.19856>.
- Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate, 2023. URL <https://arxiv.org/abs/2305.19118>.
- Wei Liu, Peijie Yu, Michele Orini, Yali Du, and Yulan He. Hunt instead of wait: Evaluating deep data research on large language models, 2026. URL <https://arxiv.org/abs/2602.02039>.
- Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. A dynamic llm-powered agent network for task-oriented agent collaboration, 2024. URL <https://arxiv.org/abs/2310.02170>.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- OpenAI. Introducing ChatGPT. <https://openai.com/blog/chatgpt>, 2022.
- OpenAI. GPT-4 technical report. 2023. URL <https://arxiv.org/abs/2303.08774>.
- Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, Michael Bendersky, Matei Zaharia, and Xing Chen. OfficeQA Pro: An enterprise benchmark for end-to-end grounded reasoning, 2026. URL <https://arxiv.org/abs/2603.08655>.
- Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023. URL <https://arxiv.org/abs/2304.03442>.
- Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, Michael Choi, Anish Agrawal, Arnav Chopra, Adam Khoja, Richard Ren, Jason Hausenloy, Oliver Zhang, Mantas Mazeika, Summer Yue, Alexandr Wang, Dan Hendrycks, et al. Humanity's last exam. 2025. URL <https://arxiv.org/abs/2501.14249>. 2,500 subject-diverse, expert-level questions across mathematics, physics, biology, humanities, computer science, and engineering. Public leaderboard at <https://agi.safe.ai/dashboard>.
- Chen Qian, Yufan Dang, Jiahao Li, Wei Liu, Zihao Xie, Yifei Wang, Weize Chen, Cheng Yang, Xin Cong, Xiaoyin Che, Zhiyuan Liu, and Maosong Sun. Experiential co-learning of software-developing agents, 2023. URL <https://arxiv.org/abs/2312.17025>.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. Chatdev: Communicative agents for software development, 2024. URL <https://arxiv.org/abs/2307.07924>.
- Daniel Shepard and Robin Salimans. AutomationBench: Evaluating agents on cross-application business workflows, 2026. URL <https://arxiv.org/abs/2604.18934>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

- Qiushi Sun, Zhangyue Yin, Xiang Li, Zhiyong Wu, Xipeng Qiu, and Lingpeng Kong. Corex: Pushing the boundaries of complex reasoning through multi-model collaboration, 2023. URL <https://arxiv.org/abs/2310.00280>.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 2024a.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024b.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL <https://arxiv.org/abs/2308.08155>.
- Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.
- Zhiheng Xi, Yiwen Ding, Wenxiang Chen, Boyang Hong, Honglin Guo, Junzhe Wang, Dingwen Yang, Chenyang Liao, Xin Guo, Wei He, Songyang Gao, Lu Chen, Rui Zheng, Yicheng Zou, Tao Gui, Qi Zhang, Xipeng Qiu, Xuanjing Huang, Zuxuan Wu, and Yu-Gang Jiang. Agentgym: Evolving large language model-based agents across diverse environments, 2024. URL <https://arxiv.org/abs/2406.04151>.
- Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks, 2024. URL <https://arxiv.org/abs/2412.14161>.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024a.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024b.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018. URL <https://arxiv.org/abs/1809.09600>.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic “differentiation” via text. *arXiv preprint arXiv:2406.07496*, 2024.
- Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet, 2025a. URL <https://arxiv.org/abs/2502.04180>.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation. In *International Conference on Learning Representations (ICLR)*, 2025b. URL <https://arxiv.org/abs/2410.10762>.
- Yinger Zhang, Shutong Jiang, Renhao Li, Jianhong Tu, Yang Su, Lianghao Deng, Xudong Guo, Chenxu Lv, and Junyang Lin. DeepPlanning: Benchmarking long-horizon agentic planning with verifiable constraints, 2026. URL <https://arxiv.org/abs/2601.18137>.

- Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. In *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024a.
- Yusen Zhang, Ruoxi Sun, Yanfei Chen, Tomas Pfister, Rui Zhang, and Sercan Ö. Arik. Chain of agents: Large language models collaborating on long-context tasks, 2024b. URL <https://arxiv.org/abs/2406.02818>.
- Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhai-long Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. MultiAgentBench: Evaluating the collaboration and competition of LLM agents, 2025. URL <https://arxiv.org/abs/2503.01935>. MARBLE framework and benchmark; code at <https://github.com/MultiagentBench/MARBLE>.

A BENCHMARK DETAILS

HumanEval (Chen et al., 2021) evaluates function-level Python code synthesis from natural-language docstrings.

MBPP (Austin et al., 2021) evaluates short-form program synthesis from natural-language descriptions.

GSM8K (Cobbe et al., 2021) evaluates multi-step reasoning over grade-school math word problems.

MATH (Hendrycks et al., 2021) evaluates competition-level mathematical reasoning across five subject areas.

HotpotQA (Yang et al., 2018) evaluates multi-hop question answering that links evidence from multiple documents.

SWE-bench Verified (Jimenez et al., 2024) evaluates repository-level software repair on human-validated GitHub issues, where the agent must produce a patch that resolves the reported bug against a hidden test suite.

DeepPlanning (Zhang et al., 2026) evaluates travel and shopping planning under user-specified constraints, and the primary metric is mean case-level accuracy across the two task families.

Humanity’s Last Exam (HLE) (Phan et al., 2025) evaluates expert-level question answering across mathematics, physics, biology, humanities, computer science, and engineering, and we use its 2,158-question text-only subset.

DevEval (Li et al., 2024) evaluates repository-level code generation under acceptance-test oracles that check whether the generated code integrates end-to-end with existing modules.

DDR-Bench (Liu et al., 2026) evaluates deep data research over 10-K SEC filings, and the overall metric averages message-wise correctness (per-question factuality) and trajectory-wise correctness (multi-turn consistency).

TheAgentCompany (Xu et al., 2024) evaluates end-to-end task completion in a simulated enterprise environment, where each task requires a full containerized stack.

MARBLE (Zhu et al., 2025) evaluates open-ended research and buyer-seller negotiation under rubric-based scores within the MultiAgentBench framework.

AutomationBench (Shepard & Salimans, 2026) evaluates cross-application business-workflow orchestration through REST APIs, and the primary metric is accuracy across its 600 tasks.

OfficeQA Pro (Opsahl-Ong et al., 2026) evaluates enterprise-grade grounded numerical reasoning over the U.S. Treasury Bulletin corpus, with correctness scored under a 1% numerical tolerance.

B CASE STUDY OF PROTOCOL EVOLUTION

Figure 6 shows how the protocol evolves across rounds on the nine long-horizon and production-oriented benchmarks, which jointly cover repository-level coding, planning, retrieval, workflow orchestration, and open-ended research. On every benchmark the number of nodes remains small (≤ 4 across rounds), so evolution acts as a rewiring process over a protocol that stays small across very different task surfaces.

The three edit levels are unevenly used. Structural edits dominate, appearing in 22 of the 27 revisions in Figure 6, and the remaining five revise a predicate or the shared context without touching the graph. DevEval is the clearest case, where the second round moves unit-test injection into the shared rule and the third round bounds the review loop and records known error patterns, both without adding a role. Both rounds change how the protocol behaves while leaving the graph fixed, which a static topology has no place to record.

On the coding and planning benchmarks, evolution mainly redistributes verification responsibility. The SWE-bench Verified protocol first attaches a retry edge from the Verifier back to the Editor, then re-routes the Verifier back to the Locator, and finally splits the Verifier into structural and semantic components. DevEval gains a self-test node between the Coder and the Reviewer. DeepPlanning

grows from a monolithic solver into a Researcher-Solver-Verifier chain with an explicit constraint checklist.

On the retrieval and research benchmarks, evolution adds evidence-preserving roles. The DDR-Bench protocol fixes the Researcher-to-Reporter handoff to forward the raw evidence set, splits a single reporting phase into two, and installs a quantitative Auditor. OfficeQA Pro converts a monolithic solver into a Reader-Solver-Verifier chain over an external document store. MARBLE replaces the initial Reviewer-Brainstormer-Writer handoff with a compact Generator-Critic-Reviser pipeline and adds a feasibility-driven retry edge from the Critic back to the Generator.

Evolution also shrinks the graph when coordination is unnecessary. AutomationBench collapses a two-node Executor-Verifier pipeline into a single-node solver with progressively tightened internal phases, while HLE grows a confidence-routed subgraph that skips verification on high-confidence answers and adds a retry edge on refused ones. Appendix C reports the compute budget these revisions consume, the candidate edits the quality screen refused, and the component measurement on which the accepted edits pay off.

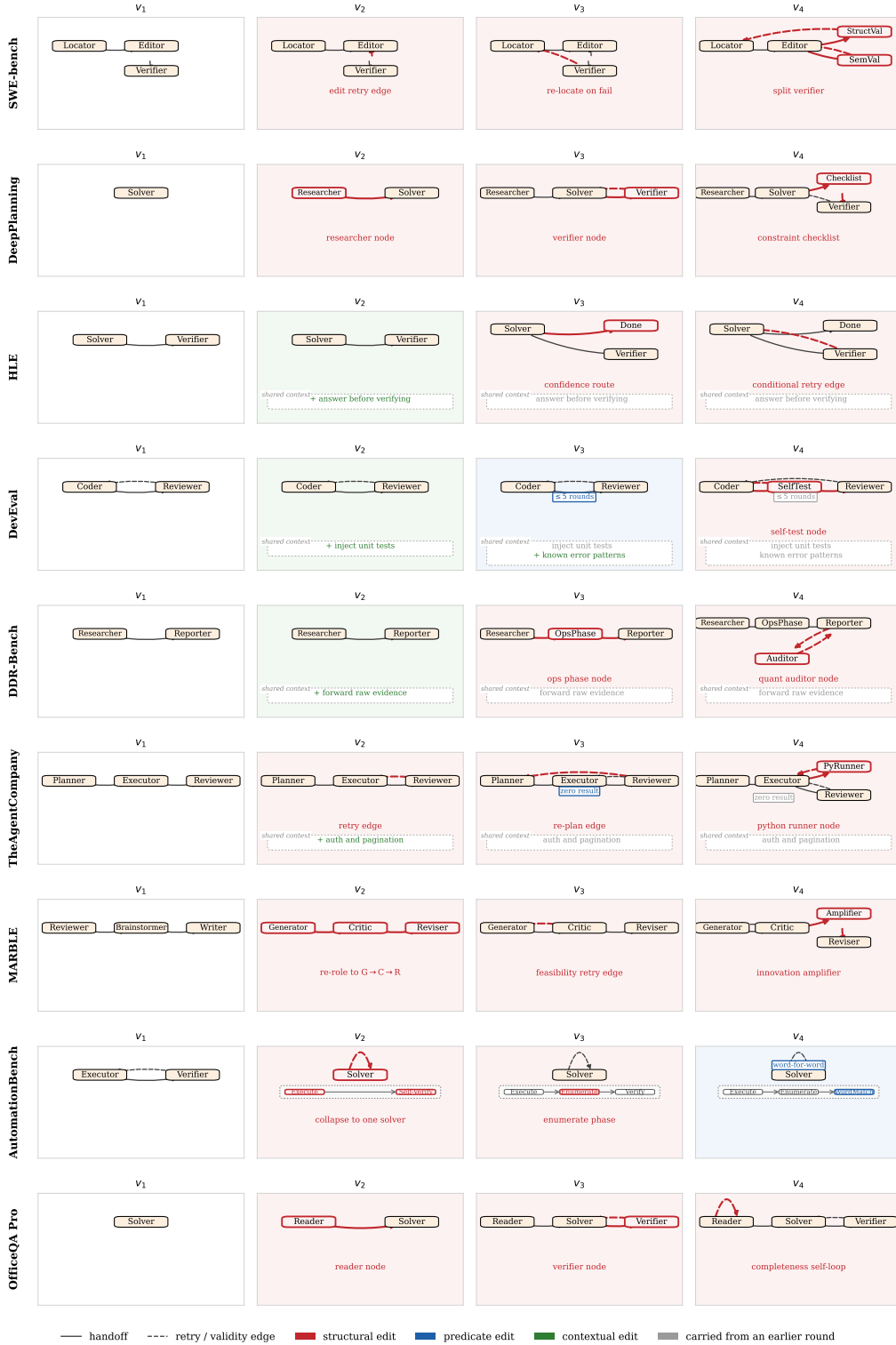


Figure 6: Protocol evolution across rounds on the nine long-horizon and production-oriented benchmarks, which form the rows. Solid ovals are agent roles, small pills inside a dashed container are internal phases of a single-agent solver, solid edges are admissible handoffs, and dashed edges are validity or retry edges. A panel shows everything the protocol carries at that round, where boxed text pinned to an element is a transition or completion predicate and the lower container holds the shared rule and culture. Elements introduced by the current round are drawn in the color of their edit level from Section 3.3, and elements carried from an earlier round are drawn in gray, so a round that leaves the graph unchanged still shows what it revised.

C ADDITIONAL ANALYSIS

C.1 COMPUTE BUDGET OF PROTOCOL EVOLUTION

Protocol evolution installs retry edges and separate verification roles, and both consume additional compute, so part of the reported gain could be bought with tokens. To relate the compute cost of evolution to its accuracy gain, we record the average number of tokens an instance consumes at every round alongside the primary metric of that round on all fourteen benchmarks. As shown in Table 3 and Figure 7, the gain does not track the budget. SWE-bench Verified absorbs 26K extra tokens for 7.4 points while DDR-Bench absorbs 34K for 32.0 points, so a comparable increment buys gains that differ by a factor of four. The ordering can also invert, since TheAgentCompany takes 20K for 7.4 points while DevEval takes only 14K for 14.8 points. What decides the outcome is which handoffs evolution keeps, so the gain reported in Section 5.1 is attributable to the protocol structure and not to a larger compute allowance.

Table 3: Average tokens per instance and the primary metric across the four evolution rounds. Each metric column is in the benchmark’s native metric and matches Table 7.

Benchmark	Tokens per instance (K)				Primary metric			
	v_1	v_2	v_3	v_4	v_1	v_2	v_3	v_4
HumanEval	9	11	12	13	98.4	100.0	100.0	100.0
MBPP	9	11	12	13	74.3	80.5	80.5	80.9
GSM8K	8	10	11	12	98.0	98.0	98.0	98.0
MATH	12	14	15	16	87.0	87.0	87.0	87.0
HotpotQA	10	12	13	14	80.7	81.9	82.3	82.3
SWE-bench Verified	32	46	54	58	78.8	85.6	85.8	86.2
DeepPlanning	22	27	31	34	44.6	44.6	54.2	54.2
HLE	18	20	22	26	33.7	35.8	35.9	37.7
DevEval	24	30	34	38	82.1	83.9	90.6	96.9
DDR-Bench	34	48	56	68	44.0	58.9	66.5	76.0
TheAgentCompany	46	55	62	66	46.3	48.8	51.2	53.7
MARBLE	20	28	33	38	73.4	91.7	95.0	98.9
AutomationBench	42	26	34	40	26.1	28.4	34.8	54.8
OfficeQA Pro	8	26	33	40	31.6	62.4	67.7	69.9

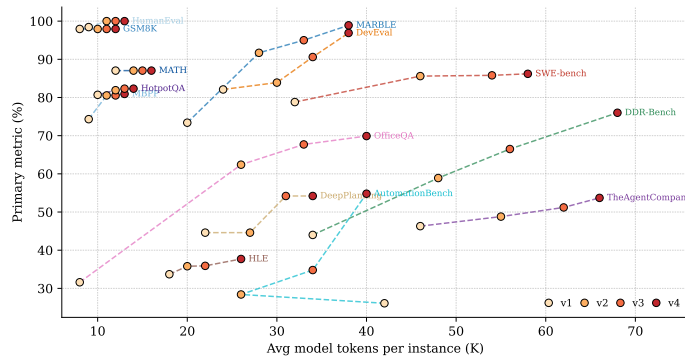


Figure 7: Primary metric against average tokens per instance across the four evolution rounds, with markers running from v_1 to v_4 in increasing darkness. The view separates the accuracy gain from the compute cost of new roles and retry edges.

C.2 CANDIDATE PROTOCOLS REFUSED BY THE QUALITY SCREEN

The quality screen of Section 3.3 deploys a candidate only when it does not degrade the realized coordination, and otherwise retains the incumbent, so the reported trajectory shows the deployed

rounds and hides every candidate the screen turned away. To recover what the screen turned away, we score every candidate proposed on SWE-bench Verified and DevEval over the complete task set of each benchmark under the same executor and tool budget, and record the decision the screen returned. As shown in Table 4, the two refused candidates are the ones that enlarge the material an existing role receives. Injecting unit tests into the reviewer loop drops DevEval acceptance from 82.1 to 75.0, and widening the discovery scope with multi-file edit instructions drops SWE-bench Verified file matching from 86.2 to 85.6. Every admitted candidate attaches or tightens a duty that can be checked, whether an extra repair round, a structural verification step, or self-verification before handoff. The screen therefore admits obligations and refuses material. Its verdict agrees with the primary metric on all ten candidates, since every admitted candidate is non-decreasing against the incumbent it replaced and both refused candidates fall below theirs, so the trace-based judgment and the benchmark metric never disagree here.

Table 4: Candidate protocols scored over the complete task set of each benchmark, in the order the evolver proposed them. SWE-bench Verified is scored by file match and DevEval by acceptance-test pass rate, and the decision column records the verdict of the quality screen.

Candidate edit	Metric (%)	Decision
<i>SWE-bench Verified, file match</i>		
Three-level context with five retries	78.8	Accepted as v_1
Larger turn and timeout allowance	85.6	Accepted as v_2
Test-path enrichment with eight retries	85.8	Accepted as v_3
Test imports mapped to candidate files	86.2	Accepted as v_4
Wider discovery with multi-file edit instructions	85.6	Refused
<i>DevEval, acceptance test</i>		
Reviewer with two repair rounds	82.1	Accepted as v_1
Unit tests injected into the reviewer loop	75.0	Refused
Five repair rounds with an error-pattern checklist	83.9	Accepted as v_2
Test-driven coder with structural verification	90.6	Accepted as v_3
Self-verification before handoff with root-cause grouping	96.9	Accepted as v_4

C.3 COMPONENT MEASUREMENTS BEHIND THE GAIN

A primary metric can improve for more than one reason, and the aggregate does not say which failure mode the evolved protocol removed. To locate the gain inside a benchmark, we decompose the SWE-bench Verified and DevEval results of every round into their component measurements. As shown in Table 5, the two benchmarks improve at different points. On SWE-bench Verified the file-match rate restricted to instances that produced a patch starts at 84.6 and ends at 86.2, while the fraction of instances that produce a patch at all rises from 79.2 to 100.0. The gain therefore comes from closing the no-output failure mode that the retry edges of Section 3.3 target. On DevEval the unit-test rate stays in a narrow band near saturation across the four rounds, while acceptance rises from 82.1 to 96.9, so the admitted edits act on the repository-level correctness that unit tests leave unchecked. In both cases the component that moves is the one the accepted edit was written to address, which is the correspondence a governed protocol exists to produce.

Table 5: Component measurements across the evolution rounds on SWE-bench Verified and DevEval. Patch applied is the fraction of instances producing a non-empty patch, file match given a patch restricts file match to those instances, and unit test and acceptance test are the two pass rates of Table 16.

Component measurement	Init	v_1	v_2	v_3	v_4
<i>SWE-bench Verified</i>					
Patch applied	79.2	93.4	100.0	100.0	100.0
File match	67.0	78.8	85.6	85.8	86.2
File match given a patch	84.6	84.4	85.6	85.8	86.2
<i>DevEval</i>					
Unit test	98.1	98.1	97.3	99.1	99.1
Acceptance test	82.1	82.1	83.9	90.6	96.9

C.4 ACTIVITY OF THE ADMISSIBLE HANDOFF SET

OpenAgency governs collaboration through the admissible handoff set $\Gamma(n, s)$, whose activity during execution determines how much of the collaboration structure is actually enforced. To verify that the boundary is active during execution, we measure how often $\Gamma(n, s)$ refuses a handoff an agent proposes. On the four benchmarks of Section 5.4 we log every proposed handoff during the v_4 runs, record the fraction $\Gamma(n, s)$ refuses, and count the runs whose outcome changes once the refusal redirects the handoff to an admissible target. As shown in Table 6, the boundary refuses 3.8% to 8.2% of proposed handoffs and changes the outcome of 2.9% to 5.3% of runs. Refusals are most frequent on TheAgentCompany, whose heterogeneous tool surface offers the most chances to address a collaborator that cannot serve the current state, and least frequent on the near-linear DDR-Bench pipeline. The boundary therefore stays active on all four benchmarks, ruling out the wrong collaborator at the points where it matters while leaving the common path unconstrained. These refused events are also the evidence the evolver consumes in Section 3.3.

Table 6: Governance boundary activity during the v_4 runs. Rejected is the percentage of proposed handoff events refused by $\Gamma(n, s)$, and Recovered is the percentage of runs whose outcome the refusal changed.

Benchmark	Rejected	Recovered
SWE-bench	6.4	4.1
DevEval	4.9	3.7
DDR-Bench	3.8	2.9
TheAgentCompany	8.2	5.3

Table 7: Primary metric across the four evolution rounds, each row in the benchmark’s native metric.

Benchmark	Single-agent	Multi-agent (init)	v1	v2	v3	v4
HumanEval	87.5	96.9	98.4	100.0	100.0	100.0
MBPP	73.5	73.5	74.3	80.5	80.5	80.9
GSM8K	97.2	97.4	98.0	98.0	98.0	98.0
MATH	12.9	32.4	87.0	87.0	87.0	87.0
HotpotQA	75.9	64.0	80.7	81.9	82.3	82.3
SWE-bench Verified	75.6	67.0	78.8	85.6	85.8	86.2
DeepPlanning	45.0	33.8	44.6	44.6	54.2	54.2
HLE	18.4	5.6	33.7	35.8	35.9	37.7
DevEval	62.5	82.1	82.1	83.9	90.6	96.9
DDR-Bench	42.8	44.0	44.0	58.9	66.5	76.0
TheAgentCompany	46.3	41.5	46.3	48.8	51.2	53.7
MARBLE	78.4	50.0	73.4	91.7	95.0	98.9
AutomationBench	23.7	23.6	26.1	28.4	34.8	54.8
OfficeQA Pro	35.3	21.1	31.6	62.4	67.7	69.9

D PER-BENCHMARK DETAILED RESULTS

This appendix reports the per-benchmark results summarized in Table 1, complementing the protocol-level traces of Appendix B. Table 7 lists the primary metric of all fourteen benchmarks across the four evolution rounds. Tables 8, 9, 10, 11, and 12 detail the general question answering benchmarks, Tables 13, 14, and 15 the long-horizon benchmarks, and Tables 16, 17, 18, 19, 20, and 21 the production-oriented benchmarks.

Table 8: Performance on HumanEval (Chen et al., 2021). The metric is pass@1.

Model / System	pass@1 (%)
Prior Ref.	92.2
OpenAgency (single-agent)	87.5
Multi-agent (init)	96.9
OpenAgency	100.0

Table 9: Performance on MBPP (Austin et al., 2021). The metric is pass@1.

Model / System	pass@1 (%)
Prior Ref.	69.3
OpenAgency (single-agent)	73.5
Multi-agent (init)	73.5
OpenAgency	80.9

Table 10: Performance on GSM8K (Cobbe et al., 2021). The metric is exact-match accuracy.

Model / System	Exact match (%)
Prior Ref.	97.6
OpenAgency (single-agent)	97.2
Multi-agent (init)	97.4
OpenAgency	98.0

Table 13: Performance on SWE-bench Verified (Jimenez et al., 2024). FM is file match, and the first three rows are reported by that work.

Model / System	%Apply	FM	%Resolved
Claude-2	—	—	4.8
GPT-4	—	—	1.7
SWE-Llama-13B	—	—	3.9
Prior Ref.	72.4	42.8	15.4
OpenAgency (single-agent)	89.8	75.6	63.8
Multi-agent (init)	79.2	67.0	56.2
OpenAgency	100.0	86.2	74.4

Table 14: Performance on DeepPlanning (Zhang et al., 2026). Avg is the mean of travel and shopping case accuracy. External rows are reported by that work.

Model / System	Avg	Tra-CS	Tra-PS	Tra-Comp	Tra-Acc	Shop-M	Shop-Acc
Grok-4.1-Fast	3.0	39.6	3.0	29.6	0.0	50.1	5.9
GLM-4.7	7.1	38.9	7.1	30.7	0.0	61.2	14.2
Qwen-Plus	7.5	37.3	7.5	25.1	0.0	63.9	15.0
Seed-1.8	11.3	43.0	11.3	45.3	0.0	68.1	22.5
Kimi-K2-Thinking	12.1	45.2	32.5	38.9	0.0	65.8	24.2
DeepSeek-V3.2	21.6	47.4	35.0	41.2	0.7	78.8	42.5
Gemini-3-Pro	23.2	58.4	25.1	41.8	0.7	78.0	45.8
o3	24.9	76.5	55.6	66.1	11.3	76.9	38.5
Claude-Sonnet-4.5	25.5	65.2	58.4	61.8	7.6	80.0	43.3
Qwen3-Max	28.7	64.0	61.7	62.8	13.8	82.6	43.5
Gemini-3-Flash	28.8	67.1	57.7	62.4	5.9	80.6	51.7
GPT-5	31.6	78.7	65.9	72.3	18.9	80.4	44.2
Claude-Opus-4.5	33.9	79.3	70.9	75.1	22.7	80.0	45.0
GPT-5.2	44.6	88.5	83.3	85.8	35.0	84.8	54.2
Qwen-Agent tool-calling (Prior Ref.)	46.9	88.8	56.7	72.7	32.9	86.8	60.8
OpenAgency (single-agent)	45.0	73.9	67.5	70.7	35.0	85.2	55.0
Multi-agent (init)	33.8	44.5	38.3	41.4	26.7	76.2	40.8
OpenAgency	54.2	—	—	—	—	—	—

Table 11: Performance on MATH (Hendrycks et al., 2021). The metric is exact-match accuracy.

Model / System	Exact match (%)
Prior Ref.	80.3
OpenAgency (single-agent)	12.9
Multi-agent (init)	32.4
OpenAgency	87.0

Table 12: Performance on HotpotQA (Yang et al., 2018). The metric is token-level F1.

Model / System	F1 (%)
Prior Ref.	76.4
OpenAgency (single-agent)	75.9
Multi-agent (init)	64.0
OpenAgency	82.3

Table 15: Performance on the Humanity’s Last Exam text-only subset (Phan et al., 2025). External rows are reported by that work.

Model / System	Accuracy (%)
GPT-4o	2.7
Grok-2	3.2
Claude-Sonnet-3.5	4.3
Gemini-1.5-Pro	5.3
Gemini-2.0-Flash-Thinking	8.1
o1	8.1
o3-mini (Prior Ref.)	18.0
Direct LLM	13.4
OpenAgency (single-agent)	18.4
Multi-agent (init)	5.6
OpenAgency	37.7

Table 16: Performance on DevEval (Li et al., 2024). UT is the unit-test pass rate and AT is the repository-level acceptance-test pass rate. The prior reference is a single coder node with no handoff, which is also the single-agent ablation.

Method	UT	AT
Direct LLM (Prior Ref.)	90.4	62.5
Multi-agent (init)	98.1	82.1
OpenAgency	99.1	96.9

Table 17: Performance on the DDR-Bench 10-K subset (Liu et al., 2026). Overall averages Message-Wise and Trajectory-Wise accuracy. External rows are reported by that work.

Model / System	Overall	Message-Wise	Trajectory-Wise
<i>Proprietary models</i>			
Gemini-2.5-Pro	20.1	24.5	15.6
Gemini-3-Flash	33.0	44.8	21.2
GPT-5.1	40.7	37.1	44.3
GPT-5-mini	42.0	46.8	37.1
GPT-5.2	43.0	44.9	41.1
Claude-Sonnet-4.5 (Prior Ref.)	69.1	77.6	60.6
<i>Open-source models</i>			
Llama-3.3-70B	6.5	9.9	3.0
Qwen2.5-72B	20.3	27.1	13.4
Qwen3-30B-A3B	28.4	42.4	14.4
MiniMax-M2	35.6	44.2	26.9
Qwen3-Next-80B-A3B	37.8	44.8	30.8
Kimi-K2	41.0	51.1	30.8
GLM-4.6	48.2	60.3	36.0
DeepSeek-V3.2	49.2	60.1	38.2
DDR reference agent	60.5	61.3	59.6
OpenAgency (single-agent)	42.8	27.3	58.2
Multi-agent (init)	44.0	–	–
OpenAgency	76.0	76.1	75.9

Table 18: Performance on TheAgentCompany (Xu et al., 2024). Score is the aggregated task score, and Steps is the average number of steps. External rows are reported by that work.

Model / System	Backbone	Success (%)	Score (%)	Steps (↓)
OpenHands	Gemini-2.5-Pro	30.3	39.1	33.5
OpenHands	Claude-Sonnet-3.7	26.3	34.7	40.9
OpenHands	Claude-Sonnet-3.5	24.0	34.4	29.2
OpenHands	Gemini-2.0-Flash	11.4	19.0	39.9
OpenHands	GPT-4o	8.6	16.7	28.7
OpenHands (Prior Ref.)	GPT-5.5	40.0	41.5	31.6
OpenAgency (single-agent)	GPT-5.5	46.7	46.3	27.4
Multi-agent (init)	GPT-5.5	–	41.5	–
OpenAgency	GPT-5.5	46.7	53.7	22.8

Table 19: Performance on MARBLE (Zhu et al., 2025). Overall averages Research and World scores. External rows are reported by that work.

Model / System	Backbone	Research (%)	World (%)	Overall (%)
Meta-Llama-3.1-8B	Llama-3.1-8B	80.9	72.8	76.9
Meta-Llama-3.1-70B	Llama-3.1-70B	80.8	72.1	76.5
Meta-Llama-3.3-70B	Llama-3.3-70B	80.0	73.2	76.6
GPT-3.5-turbo	GPT-3.5-turbo	70.2	71.7	71.0
GPT-4o-mini (Prior Ref.)	GPT-4o-mini	84.1	74.5	79.3
OpenAgency (single-agent)	GPT-5.5	66.8	90.0	78.4
Multi-agent (init)	GPT-5.5	–	–	50.0
OpenAgency	GPT-5.5	97.8	100.0	98.9

Table 20: Performance on AutomationBench (Shepard & Salimans, 2026). External rows are reported by that work.

Model / System	Acc (%)
GPT-6-Sol	33.2
GPT-6-Astra	41.4
Claude-Opus-5.5 (Prior Ref.)	42.5
OpenAgency (single-agent)	23.7
Multi-agent (init)	23.6
OpenAgency	54.8

Table 21: Performance on OfficeQA Pro (Opsahl-Ong et al., 2026). External rows are reported by that work.

Model / System	Corpus format	Acc@1% (%)
GPT-5-mini	Full PDF	30.8
GPT-5.4	Full PDF	36.1
Gemini-3.1-Pro	Full PDF	42.9
GPT-5.1	Full PDF	45.1
Claude-Opus-4.5	Full PDF	46.6
Claude-Opus-4.6 (Prior Ref.)	ai_parse_document	57.1
Direct LLM	Full PDF	26.3
OpenAgency (single-agent)	Full PDF	35.3
Multi-agent (init)	Full PDF	21.1
OpenAgency	Full PDF	69.9